

ESP32-Based High-Speed Line Follower Robot



Figure 1: Assembled ESP32 line follower robot

1. Project Overview

This project is an autonomous high-speed line-following robot built around the **ESP32 DevKit** microcontroller. The robot detects and tracks a contrasting line (typically black on white) using a reflectance sensor array and maintains stable trajectory control with a **PD (Proportional-Derivative)** control algorithm.

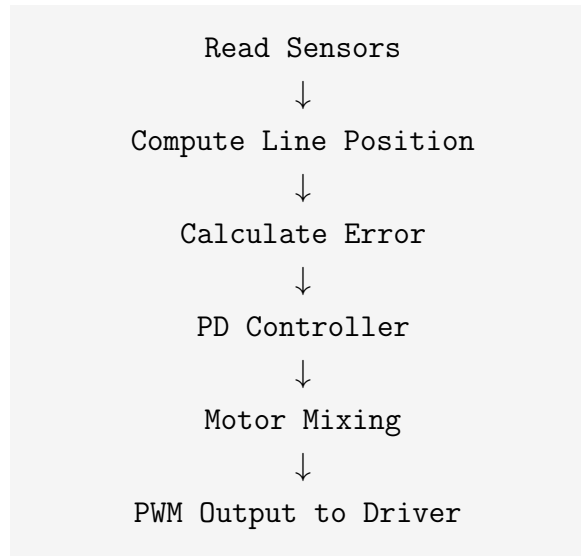
The system is optimised for:

- Fast closed-loop control
- Stable high-speed cornering
- Clean hardware abstraction
- Reliable power regulation
- Competition-ready robustness

Unlike a manually controlled RC vehicle, this robot operates as a fully autonomous closed-loop control system — continuously sensing its environment and dynamically correcting its trajectory.

2. System Architecture

The control pipeline follows a strict layered structure:



This structure ensures separation of sensing and actuation, clean debugging and tuning, maintainable firmware design, and deterministic execution timing.

3. Core Hardware Components

3.1 Microcontroller — ESP32 DevKit (ESP32-WROOM)

The ESP32 reads the QTR-8RC sensor array, computes the weighted line position, runs the PD control loop, generates high-frequency PWM signals, and drives the motor driver. It was selected for its high processing speed, hardware PWM (LEDC) support, multiple GPIOs for parallel sensor reading, and superior performance over traditional 8-bit boards.

3.2 Line Detection — QTR-8RC Reflectance Sensor Array

An 8-channel reflectance sensor array detects contrast between the line and background. Each sensor measures surface reflectivity, enabling computation of a weighted average line position across the range:

Value	Position
0	Far left
3500	Centre
7000	Far right

This provides sub-sensor resolution and a smooth steering response.

3.3 Motor Driver — Cytron MDD3A Dual Channel

- Independent left/right motor channels
- High current capability
- Direction control via PWM polarity
- PWM frequency ≈ 10 kHz (reduces audible noise)

3.4 Drivetrain — Differential Drive

Two independently driven DC gear motors and one passive caster provide steering by varying left and right motor speeds. Sharp turns use pivot logic (one motor forward, one backward).

3.5 Power System

- **Source:** 3S LiPo battery (11.1 V nominal)
- **Regulation:** LM2596-based DC–DC buck converter
- LiPo → Motor driver (direct supply)
- LiPo → LM2596 → ESP32 (regulated supply)

A common ground is maintained across all components to ensure signal integrity.

4. Control Strategy

4.1 Error Definition

$$\text{Error} = \text{CenterPosition} - \text{MeasuredPosition}$$

where the centre position is typically 3500.

4.2 PD Controller

$$\text{Correction} = K_p \cdot \text{Error} + K_d \cdot \frac{d(\text{Error})}{dt}$$

- K_p — Proportional gain: corrects present deviation.
- K_d — Derivative gain: damps oscillation and overshoot.

4.3 Motor Mixing

$$\text{Left} = \text{BaseSpeed} + \text{Correction}$$

$$\text{Right} = \text{BaseSpeed} - \text{Correction}$$

Outputs are clamped to ± 255 and mapped to PWM duty cycles.

5. Tuned Control & Motion Parameters

Table 1: Control and motion parameter summary

Parameter	Variable	Value
PD Controller		
Proportional gain	K_p	0.018
Derivative gain	K_d	1.500
Base speed	baseSpeed	55
Deadband	deadband	150
Filter coefficient	α	0.40
Corner Recovery		
Pivot duration	pivotMs	350 ms
Pivot PWM	pivotPwm	90
Post-bump duration	postBumpMs	150 ms
Bump PWM	bumpPwm	65
Corner Detection		
Confirm count	cornerCount	5
Detect threshold	cornerThreshold	700
Motor Trim		
Left trim	leftTrim	0
Right trim	rightTrim	-10

Parameter Notes

K_p (**0.018**) Controls the immediate reaction to position error. Increasing it improves responsiveness but may introduce oscillation.

K_d (**1.500**) Provides damping by reacting to the rate of change of error, stabilising high-speed motion and reducing overshoot.

baseSpeed (**55**) Nominal forward velocity before correction is applied.

deadband (**150**) Small error range where no correction is applied, preventing jitter from sensor noise.

α (**0.40**) Low-pass filter coefficient balancing responsiveness and noise suppression.

pivotMs (**350 ms**) Duration of in-place rotation during sharp corner recovery.

pivotPwm (90)

Motor power used during pivot turns.

postBumpMs (150 ms)

Forward movement duration after detecting a junction or recovering from a corner.

bumpPwm (65)

Motor power applied during post-corner stabilisation.

cornerCount (5)

Consecutive strong detections required to confirm a corner condition.

cornerThreshold (700)

Sensor value threshold for detecting a strong black surface (corner ID).

leftTrim / rightTrim (0 / -10)

Calibration offsets compensating for mechanical differences between motors to ensure straight-line motion.

6. Design Philosophy

The firmware and hardware were engineered around the following principles:

- Non-blocking firmware (no delay-based timing)
- High PWM frequency to reduce motor noise
- Layered control architecture with clear separation of sensing and actuation
- Pivot-based corner recovery for reliable sharp-turn handling
- Stable regulated power supply for logic reliability

The result is a compact, fast, and robust line-following platform suitable for competitive robotics applications.